

实验 4：频率隐藏保序加密（FH-OPE）

数据安全第四次实验报告

姓名：郭佳成 学号：2311990 专业：密码科学与技术
代码：<https://github.com/Fighting05/DataSecurity>

2026 年 5 月 29 日

目录

1	实验要求	3
2	实验环境	3
3	实验原理	3
3.1	保序加密（OPE）的基本思想	3
3.2	FH-OPE 的频率隐藏机制	3
3.3	编码树的整体结构	4
3.4	插入流程详解	5
3.5	编码区间分配机制（Encode 与 Recode）	5
3.6	叶节点分裂过程	5
4	代码实现	6
4.1	项目文件结构	6
4.2	节点类的设计（Node.h）	6
4.3	中间节点的位置路由（InternalNode::insert）	7
4.4	叶节点编码生成（LeafNode::Encode 与 Recode）	8
4.5	MySQL 存储过程（example.sql）	10
4.6	客户端核心逻辑（client.py）	10
4.6.1	CalPos：频率隐藏的关键	10
4.6.2	随机加密：保证密文不可区分	11

5	实验步骤与运行结果	11
5.1	环境安装	11
5.2	编译动态链接库	13
5.3	初始化数据库	13
5.4	运行客户端	14
5.5	重复插入同一数值的测试	15
6	算法复杂度	17
7	安全性分析	17
7.1	频率隐藏性	17
7.2	保序性	17
7.3	前向安全性	17
8	遇到的问题 and 解决	17
9	总结	18

1 实验要求

本实验要求参照教材 7.3.3 节的内容，动手实现一个频率隐藏 OPE (FH-OPE) 方案。具体包含三部分工作：

1. 按照教材描述，实现编码树的 C++ 核心代码 (Node.h、Node.cpp) 和 MySQL UDF 接口 (UDF.cpp)，将编码逻辑以动态链接库的形式注入数据库。
2. 实现 Python 客户端 (client.py)，完成客户端本地表维护、AES 随机加密、插入/搜索功能。
3. 对客户端程序进行扩展测试，连续插入相同数值多次，观察编码树的分裂过程和编码更新情况，验证频率隐藏效果。

2 实验环境

- 操作系统：Windows 11 (WSL2 Ubuntu 22.04 用于编译和运行)
- 数据库：MySQL 8.0 (WSL2 内)
- 开发语言：Python 3.10、C++17
- C++ 依赖：libmysqlclient-dev、OpenSSL (-lcrypto)
- Python 依赖：pycryptodome、pymysql

3 实验原理

3.1 保序加密 (OPE) 的基本思想

保序加密 (Order-Preserving Encryption, OPE) 是一种特殊的对称加密方案，其核心性质为：对于任意两个明文 $m_1 < m_2$ ，加密后的密文满足 $\text{Enc}(m_1) < \text{Enc}(m_2)$ 。这个性质使得服务器在不解密的情况下，仅凭密文大小就能完成范围查询，对密文数据库非常实用。

然而传统 OPE 是确定性的，即同一个明文每次加密得到的密文完全相同。这带来了严重的安全隐患：攻击者只需统计密文出现频率，就能将高频密文与高频明文进行对应，进而大幅缩小明文空间。

3.2 FH-OPE 的频率隐藏机制

频率隐藏保序加密 (Frequency-Hiding OPE, FH-OPE) 在保持保序性的同时，打破了确定性约束。其核心思想是：同一个明文每次插入时，都被分配到一个随机选取的

编码位置，使得密文不再一一对应某个明文值。

以下面这张表为例：

明文	1	4	4	5	5	7
编码（密文大小）	13	14	16	17	19	21

两个”4” 分别被编码为 14 和 16，两个”5” 分别被编码为 17 和 19。攻击者看到的是 6 个各不相同的密文，无法区分哪些密文来自同一个明文，频率信息因此被隐藏。

3.3 编码树的整体结构

本实验采用的是 2021 年 Li 等人提出的无交互 FH-OPE 方案，核心数据结构是一棵改进的 B+ 树，称为**编码树**。下图展示了编码树的整体组织方式：

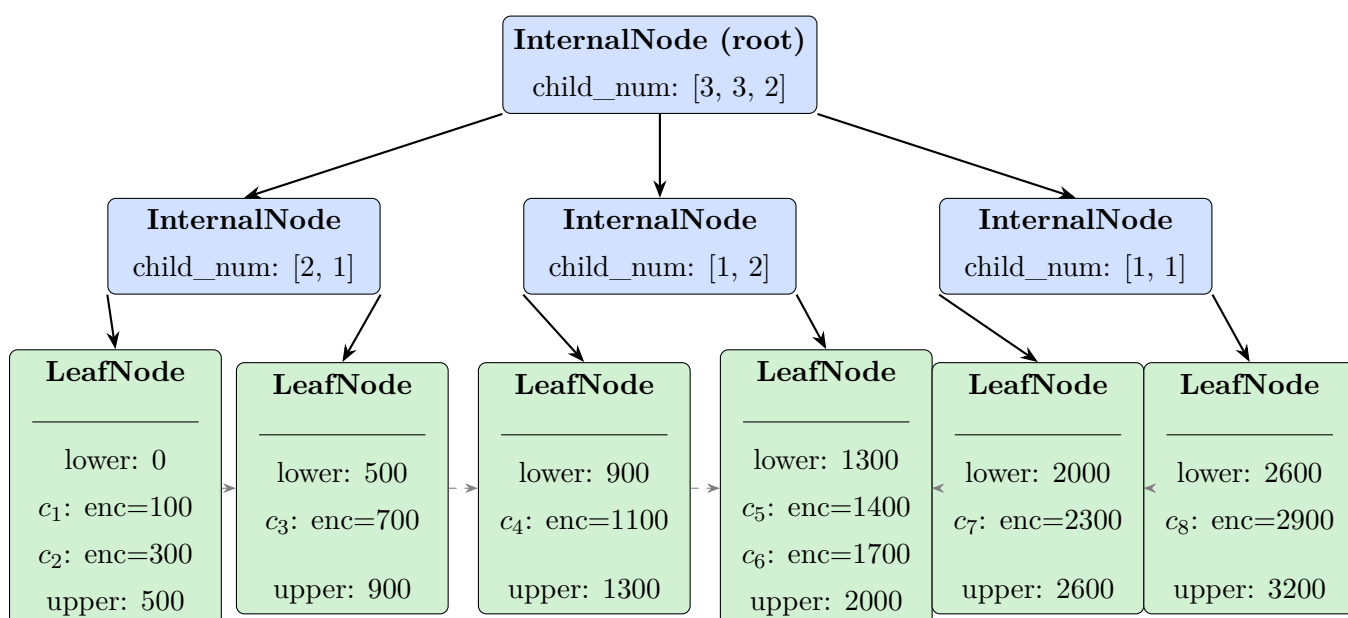


图 1: 编码树整体结构示意图

编码树有以下几个关键设计：

- **中间节点 (InternalNode)** 不存储具体的密文或关键词，只记录每棵子树中有多少条密文 (`child_num` 数组)。对 `child_num` 做前缀和即可定位任意全局位置对应的叶节点，实现高效路由。
- **叶节点 (LeafNode)** 才真正存放密文列表 (`cipher`) 和对应的编码列表 (`encoding`)。每个叶节点维护一个编码区间 `[lower, upper)`，节点内所有编码都落在这个区间内。
- **叶节点之间通过双向链表连接** (`left_bro/right_bro`)，便于重编码时向邻居借用区间空间。

3.4 插入流程详解

插入一条新密文分为客户端和服务端两个部分：

1. **客户端计算插入位置 (CalPos)**: 客户端维护本地计数表 `local_table`，记录每个明文出现了多少次。设明文为 m ，则其合法插入范围是 $[\text{presum}(m), \text{presum}(m) + \text{count}(m) - 1]$ ，客户端在此范围内随机取一个位置。
2. **客户端随机加密 (Random_Encrypt)**: 用随机 IV 对明文进行 AES-CBC 加密，相同明文每次得到不同密文。
3. **服务端路由 (InternalNode::insert)**: 从根节点出发，逐层对 `child_num` 做累减，将全局 `pos` 转换为叶节点内的局部偏移。
4. **叶节点生成编码 (LeafNode::Encode)**: 取新密文左右相邻编码的中点作为新编码；若区间不足则触发重编码 (Recode)。
5. **节点满则分裂 (LeafNode::rebalance)**: 若叶节点密文数达到阈值 $M = 128$ ，后半部分密文移入新节点，父节点的 `child_num` 随之更新。

3.5 编码区间分配机制 (Encode 与 Recode)

下图展示了 `LeafNode::Encode` 的工作原理——取左右邻居编码的区间中点：

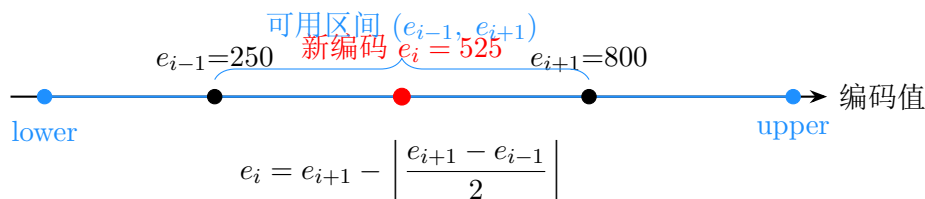


图 2: `LeafNode::Encode` 编码分配示意图

当可用区间不足 ($e_{i+1} - e_{i-1} < 2$) 时，触发 **Recode**：将当前叶节点（或向邻居扩展后的节点组）的所有编码在总区间内均匀重分配，间隔为 $\lfloor (\text{upper} - \text{lower}) / \text{count} \rfloor$ 。重编码期间 `update` 映射表记录所有需要同步到数据库的 (`cipher` $\rightarrow$ `new_encoding`) 对，由 MySQL 存储过程统一更新。

3.6 叶节点分裂过程

下图展示了叶节点密文数达到阈值 $M = 128$ 时发生分裂的前后状态：

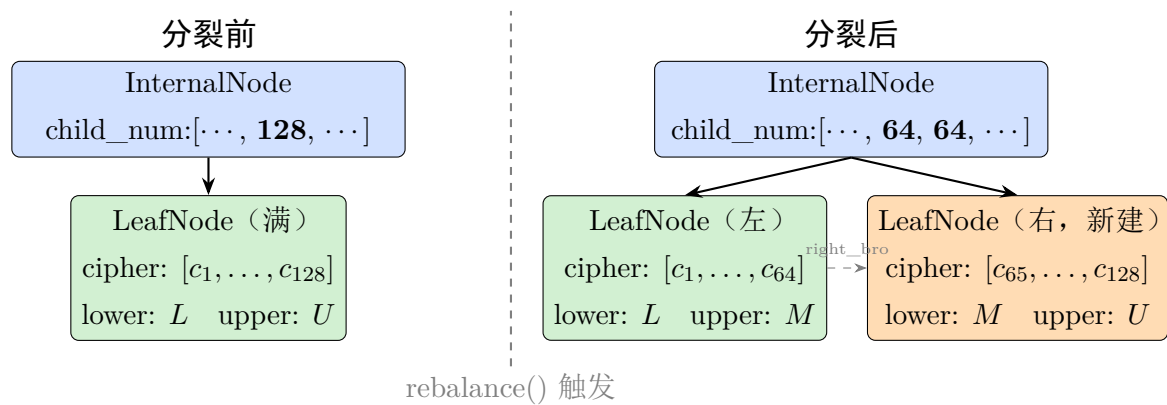


图 3: 叶节点分裂 (LeafNode::rebalance) 示意图

分裂时新叶节点的 lower 设为原节点后半段第一个编码值，原节点 upper 缩短至此，两节点的编码区间自然分开，已有编码无需重算。

4 代码实现

4.1 项目文件结构

文件	作用
Node.h	三个节点类的声明及全局变量
Node.cpp	编码树的完整实现（插入、编码、分裂、重编码）
UDF.cpp	MySQL UDF 接口，暴露给数据库的 5 个函数
client.py	Python 客户端（本地表、AES 加密、插入/搜索）
example.sql	建表、注册 UDF 函数、创建存储过程

4.2 节点类的设计 (Node.h)

三个类通过虚函数实现多态，InternalNode 和 LeafNode 各自实现 insert 和 search。

```

1 class Node {
2 public:
3     int type;           // 1=LeafNode, 2=InternalNode
4     int parent_index;   // 当前节点是父节点的第几个孩子
5     Node *parent = NULL;
6     virtual long long insert(int pos, string cipher) { return 0; };
7     virtual long long search(int pos) { return 0; };
8 };
9

```

```
10 class InternalNode : public Node {
11 public:
12     vector<int>    child_num; // 每棵子树的密文计数
13     vector<Node*> child;      // 子节点指针
14     long long insert(int pos, string cipher) override;
15     long long search(int pos) override;
16     void insert_node(int index, Node *new_node);
17     void rebalance() override;
18 };
19
20 class LeafNode : public Node {
21 public:
22     vector<string>    cipher; // 密文列表
23     vector<long long> encoding; // 编码列表
24     LeafNode *left_bro, *right_bro;
25     long long lower = -1, upper = -1; // 编码区间
26     long long Encode(int pos);
27     void rebalance() override;
28     long long insert(int pos, string cipher) override;
29     long long search(int pos) override;
30 };
31 const int M = 128; // 叶节点密文容量上限
```

Listing 1: Node.h 节点类声明

4.3 中间节点的位置路由 (InternalNode::insert)

中间节点的 insert 是树的”导航层”。它通过累减 child_num 把全局位置 pos 转换为某个子节点的内部偏移，然后递归调用该子节点的 insert。

```
1 long long InternalNode::insert(int pos, string cipher)
2 {
3     for (int i = 0; i < this->child.size(); i++)
4     {
5         if (pos > this->child_num.at(i))
6         {
7             pos = pos - this->child_num.at(i); // 跳过该子树
8         }
9         else
10        {
11            this->child_num.at(i)++; // 更新本子树计数
```

```
12         return this->child.at(i)->insert(pos, cipher);
13     }
14 }
15 // 所有子树都跳过则插入最后一个
16 this->child_num.back()++;
17 return this->child.back()->insert(pos, cipher);
18 }
```

Listing 2: InternalNode::insert 位置路由

以 $\text{child_num} = [3, 3, 2]$ 、插入全局位置 $\text{pos} = 5$ 为例：第一棵子树有 3 个密文， $5 > 3$ ，减去后 $\text{pos} = 2$ ；进入第二棵子树， $2 \leq 3$ ，将 $\text{child_num}[1]$ 加 1，递归调用第二棵子树的 $\text{insert}(2, \text{cipher})$ 。

4.4 叶节点编码生成 (LeafNode::Encode 与 Recode)

找到叶节点内的偏移后，调用 Encode 为新密文分配编码：

```
1 long long LeafNode::Encode(int pos)
2 {
3     long long left  = this->lower;
4     long long right = this->upper;
5
6     if (pos > 0)
7         left  = this->encoding.at(pos - 1); // 左邻居编码
8     if (pos < (int)this->encoding.size() - 1)
9         right = this->encoding.at(pos + 1); // 右邻居编码
10
11     if (floor(right - left) < 2)
12     {
13         // 区间不足，触发重编码
14         vector<LeafNode*> node_list;
15         node_list.push_back(this);
16         Recode(node_list);
17         return 0; // 0 通知调用方需要同步数据库
18     }
19     else
20     {
21         long long frag = (right - left) / 2;
22         this->encoding.at(pos) = right - frag;
23         return this->encoding.at(pos);
24     }
```


25 }

Listing 3: LeafNode::Encode 编码分配

当可用区间不足时，Recode 将节点组的所有编码均匀重分配。若总区间仍不足，则向左右兄弟扩展后递归：

```

1 void Recode(vector<LeafNode*> node_list)
2 {
3     long long lb = node_list.front()->lower;
4     long long rb = node_list.back()->upper;
5     int total = 0;
6     for (auto n : node_list) total += n->cipher.size();
7
8     if ((rb - lb) > total)
9     {
10         start_update = lb; end_update = rb;
11         long long frag = (rb - lb) / total;
12         long long cd = lb;
13         for (auto node : node_list) {
14             node->lower = cd;
15             for (int j = 0; j < (int)node->encoding.size(); j++) {
16                 node->encoding[j] = cd;
17                 update[node->cipher[j]] = cd; // 记录待同步到DB的映射
18
19                 cd += frag;
20             }
21             node->upper = cd;
22         }
23         node_list.back()->upper = rb;
24     }
25     else
26     {
27         // 区间不足，向左/右兄弟扩展后递归
28         if (node_list.front()->left_bro)
29             node_list.insert(node_list.begin(), node_list.front()->
30 left_bro);
31         if (node_list.back()->right_bro)
32             node_list.push_back(node_list.back()->right_bro);
33         else
34             node_list.back()->upper *= 2;

```

```
33     Recode(node_list);
34 }
35 }
```

Listing 4: Recode 均匀重分配编码（核心逻辑）

4.5 MySQL 存储过程 (example.sql)

插入操作封装在存储过程 `pro_insert` 中，保证 C++ 编码树内存状态与数据库表的一致性：

```
1 CREATE PROCEDURE pro_insert(IN pos INT, IN ct VARCHAR(512))
2 BEGIN
3     DECLARE i BIGINT DEFAULT 0;
4     SET i = FHInsert(pos, ct);    -- 调用C++ UDF，返回新编码（0表示触
   发了Recode）
5     INSERT INTO example VALUES (i, ct);
6     IF i = 0 THEN
7         -- 同步Recode影响的编码到数据库
8         UPDATE example
9         SET encoding = FHUpdate(ciphertext)
10        WHERE (encoding >= FHStart() AND encoding < FHEnd())
11            OR (encoding = 0);
12     END IF;
13 END
```

Listing 5: pro_insert 存储过程

4.6 客户端核心逻辑 (client.py)

4.6.1 CalPos: 频率隐藏的关键

```
1 local_table = {} # 本地计数表，记录每个明文出现次数
2
3 def CalPos(plaintext):
4     # presum = 所有小于 plaintext 的明文出现次数之和
5     presum = sum([v for k, v in local_table.items() if k < plaintext
6 ])
7
8     if plaintext in local_table:
9         local_table[plaintext] += 1
```

```
9      # 在 [presum, presum+count-1] 内随机选位置
10     # 频率隐藏核心：相同明文 -> 不同随机位置 -> 不同编码
11     return random.randint(presum, presum + local_table[plaintext]
12     - 1)
13 else:
14     local_table[plaintext] = 1
15     return presum # 首次出现，直接放在区间左边界
```

Listing 6: client.py CalPos 函数

4.6.2 随机加密：保证密文不可区分

```
1 key      = get_random_bytes(16)
2 base_iv  = get_random_bytes(16)
3
4 def Random_Encrypt(plaintext):
5     iv      = get_random_bytes(16)          # 每次使用全新随机 IV
6     inner   = AES_ENC(plaintext.encode('utf-8'), iv)
7     outer   = AES_ENC(iv + inner, base_iv)   # 外层把 IV 也一并加密
8     return base64.b64encode(outer).decode('utf-8')
9
10 def Random_Decrypt(ciphertext):
11     outer   = AES_DEC(base64.b64decode(ciphertext.encode('utf-8')),
12     base_iv)
13     iv      = outer[:16]                    # 从密文头部取回 IV
14     inner   = AES_DEC(outer[16:], iv)
15     return inner.decode('utf-8')
```

Listing 7: client.py 随机加密与解密

两层 AES-CBC：外层用固定 base_iv 封装随机 IV；内层用随机 iv 加密明文。由于每次 iv 不同，同一明文每次加密结果完全不同。

5 实验步骤与运行结果

5.1 环境安装

在 WSL（Ubuntu 22.04）中安装所需软件包：

```
1 sudo apt update
2 sudo apt install mysql-server libmysqlclient-dev
```

```
3 sudo service mysql start
4 sudo apt install python3-pycryptodome python3-pymysql
```

```
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全$ cd /mnt/d/大三下/数据安全/Lab4/FH-OPE
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$ sudo apt install libmysqlclient-dev
[sudo] password for fighting:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libmysqlclient21 libzstd-dev mysql-common
The following NEW packages will be installed:
  libmysqlclient-dev libmysqlclient21 libzstd-dev mysql-common
0 upgraded, 4 newly installed, 0 to remove and 188 not upgraded.
Need to get 3216 kB of archives.
After this operation, 17.3 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 mysql-common all 5.8+1.1.0build1 [6746 B]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libmysqlclient21 amd64 8.0.45-0ubuntu0.24.04.1 [1255 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libzstd-dev amd64 1.5.5+dfsg2-2build1.1 [364 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libmysqlclient-dev amd64 8.0.45-0ubuntu0.24.04.1 [1591 kB]
Fetched 3216 kB in 6s (547 kB/s)
Selecting previously unselected package mysql-common.
(Reading database ... 79677 files and directories currently installed.)
Preparing to unpack .../mysql-common_5.8+1.1.0build1_all.deb ...
Unpacking mysql-common (5.8+1.1.0build1) ...
Selecting previously unselected package libmysqlclient21:amd64.
Preparing to unpack .../libmysqlclient21_8.0.45-0ubuntu0.24.04.1_amd64.deb ...
Unpacking libmysqlclient21:amd64 (8.0.45-0ubuntu0.24.04.1) ...
Selecting previously unselected package libzstd-dev:amd64.
Preparing to unpack .../libzstd-dev_1.5.5+dfsg2-2build1.1_amd64.deb ...
Unpacking libzstd-dev:amd64 (1.5.5+dfsg2-2build1.1) ...
Selecting previously unselected package libmysqlclient-dev.
Preparing to unpack .../libmysqlclient-dev_8.0.45-0ubuntu0.24.04.1_amd64.deb ...
Unpacking libmysqlclient-dev (8.0.45-0ubuntu0.24.04.1) ...
Setting up mysql-common (5.8+1.1.0build1) ...
update-alternatives: using /etc/mysql/my.cnf.fallback to provide /etc/mysql/my.cnf (my.cnf) in auto mode
Setting up libmysqlclient21:amd64 (8.0.45-0ubuntu0.24.04.1) ...
Setting up libzstd-dev:amd64 (1.5.5+dfsg2-2build1.1) ...
Setting up libmysqlclient-dev (8.0.45-0ubuntu0.24.04.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.6) ...
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$
```

图 4: 安装 MySQL 及开发库

```

reading /usr/share/mecab/dic/ipadic/Verb.csv ... 130750
reading /usr/share/mecab/dic/ipadic/Noun.others.csv ... 151
reading /usr/share/mecab/dic/ipadic/Noun.csv ... 60477
reading /usr/share/mecab/dic/ipadic/Noun.adverbial.csv ... 795
reading /usr/share/mecab/dic/ipadic/Noun.proper.csv ... 27328
reading /usr/share/mecab/dic/ipadic/Noun.verbal.csv ... 12146
emitting double-array: 100% |#####|
reading /usr/share/mecab/dic/ipadic/matrix.def ... 1316x1316
emitting matrix : 100% |#####|

done!
update-alternatives: using /var/lib/mecab/dic/ipadic-utf8 to provide /var/lib/mecab/dic/debian (mecab-dictionary) in au
Setting up mysql-server-8.0 (8.0.45-0ubuntu0.24.04.1) ...
update-alternatives: using /etc/mysql/mysql.cnf to provide /etc/mysql/my.cnf (my.cnf) in auto mode
Renaming removed key_buffer and myisam-recover options (if present)
mysqld will log errors to /var/log/mysql/error.log
mysqld is running as pid 7257
Created symlink /etc/systemd/system/multi-user.target.wants/mysql.service → /usr/lib/systemd/system/mysql.service.
Setting up mysql-server (8.0.45-0ubuntu0.24.04.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$ sudo apt install mysql-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mysql-server is already the newest version (8.0.45-0ubuntu0.24.04.1).
0 upgraded, 0 newly installed, 0 to remove and 84 not upgraded.
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$ sudo service mysql start
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$

```

图 5: 安装并启动 MySQL 服务

5.2 编译动态链接库

编码树核心逻辑以 MySQL UDF 形式提供，需编译为 .so 共享库并部署到 MySQL 插件目录：

```

1 cd /mnt/d/大三下/数据安全/Lab4/FH-OPE
2
3 # -shared -fPIC 生成位置无关共享库；-lcrypto 链接 OpenSSL
4 g++ -shared -fPIC UDF.cpp Node.cpp -lcrypto -o libfhope.so
5
6 sudo cp libfhope.so /usr/lib/mysql/plugin/

```

```

fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$ g++ -shared -fPIC UDF.cpp Node.cpp -lcrypto -o libfhope.so
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$ sudo cp libfhope.so /usr/lib/mysql/plugin/
fighting@LAPTOP-DF52FLOB:/mnt/d/大三下/数据安全/Lab4/FH-OPE$

```

图 6: 编译并部署 UDF 动态链接库

5.3 初始化数据库

登录 MySQL，创建数据库并导入 SQL 脚本（建表、注册 UDF 函数、创建存储过程）：

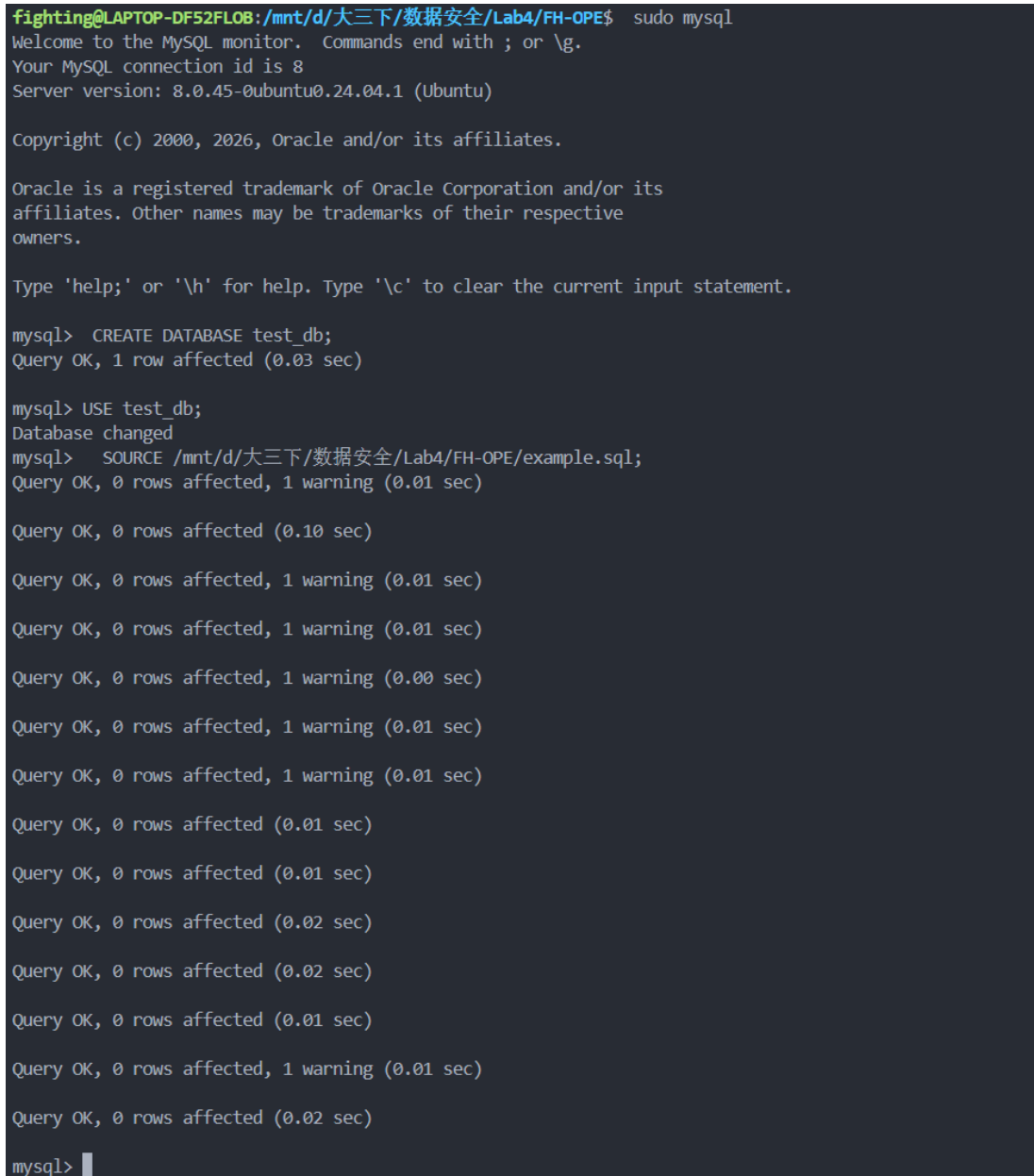
```

1 sudo mysql -u root -p
2 mysql> CREATE DATABASE test_db;

```

```
3 mysql> USE test_db;  
4 mysql> SOURCE /mnt/d/大三下/数据安全/Lab4/FH-OPE/example.sql;
```

导入成功后,数据库中会有:表 `example(encoding bigint,ciphertext varchar(512))`; 函数 `FHInsert`、`FHSearch`、`FHUpdate`、`FHStart`、`FHEnd`; 以及存储过程 `pro_insert`。



```
fighting@LAPTOP-DF52FL08:/mnt/d/大三下/数据安全/Lab4/FH-OPE$ sudo mysql  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 8  
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)  
  
Copyright (c) 2000, 2026, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql> CREATE DATABASE test_db;  
Query OK, 1 row affected (0.03 sec)  
  
mysql> USE test_db;  
Database changed  
mysql> SOURCE /mnt/d/大三下/数据安全/Lab4/FH-OPE/example.sql;  
Query OK, 0 rows affected, 1 warning (0.01 sec)  
  
Query OK, 0 rows affected (0.10 sec)  
  
Query OK, 0 rows affected, 1 warning (0.01 sec)  
  
Query OK, 0 rows affected, 1 warning (0.01 sec)  
  
Query OK, 0 rows affected, 1 warning (0.00 sec)  
  
Query OK, 0 rows affected, 1 warning (0.01 sec)  
  
Query OK, 0 rows affected, 1 warning (0.01 sec)  
  
Query OK, 0 rows affected (0.01 sec)  
  
Query OK, 0 rows affected (0.01 sec)  
  
Query OK, 0 rows affected (0.02 sec)  
  
Query OK, 0 rows affected (0.02 sec)  
  
Query OK, 0 rows affected (0.01 sec)  
  
Query OK, 0 rows affected, 1 warning (0.01 sec)  
  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> █
```

图 7: 创建数据库并导入 SQL 脚本

5.4 运行客户端

```
1 cd /mnt/d/大三下/数据安全/Lab4/FH-OPE  
2 python3 client.py
```

```

fighting@LAPTOP-DF52FLO8:/mnt/d/大三下/数据安全/Lab4/FH-OPE$ python3 /mnt/d/大三下/数据安全/Lab4/FH-OPE/client.py
=====
测试：连续插入相同数值 'apple' 5次
=====

--- 第 1 次插入 'apple' ---
插入明文：'apple'，计算位置：0，密文：Hcfn1KCSy0ubNagy6KsEp5r5dT1uQ8...

--- 第 2 次插入 'apple' ---
插入明文：'apple'，计算位置：0，密文：dtNLseaValuwSP8msUeS9CuHPPKa77...

--- 第 3 次插入 'apple' ---
插入明文：'apple'，计算位置：0，密文：zMHpJLZgA3i3+0eXNFtRtW6cGyBbGF...

--- 第 4 次插入 'apple' ---
插入明文：'apple'，计算位置：1，密文：zlxUng34piAhDDO+F6sUjp5ixyjTFy...

--- 第 5 次插入 'apple' ---
插入明文：'apple'，计算位置：4，密文：WgqozGf05e3/ZyannlwOSF0UgcyDkC...

=====
测试：连续插入相同数值 'banana' 3次
=====

--- 第 1 次插入 'banana' ---
插入明文：'banana'，计算位置：5，密文：hfkdtJJaC0c0XkLs2ZJDup8tY+pwZBN...

--- 第 2 次插入 'banana' ---
插入明文：'banana'，计算位置：6，密文：toCHDillxREBWJIZLunViiFaPDEMNz...

--- 第 3 次插入 'banana' ---
插入明文：'banana'，计算位置：7，密文：D3HgEKRAUkWhbNUUxcfkNDCPB6stL9u...

=====
测试：混合插入不同数值
=====
插入明文：'apple'，计算位置：5，密文：bDYI9VCJewva5qD59kMuLy+5wmqBOG...
插入明文：'pear'，计算位置：9，密文：pdPw/OvK8WgSbK/lzr7hj0oLvFhK4k...
插入明文：'banana'，计算位置：6，密文：cLTm8KWCWEtJgsemCtoB59IghlFuB...
插入明文：'orange'，计算位置：10，密文：Mp/29QFkJY7q2i4PAsmofyrUKTTmYe...
插入明文：'cherry'，计算位置：10，密文：n5sFN1LzguBpox1xED9qPed0l8UrHu...
插入明文：'apple'，计算位置：4，密文：jLYdAcoK/R+w24gTj6apjo+3yyp65A...
插入明文：'cherry'，计算位置：12，密文：BA9I2NV/jAMcd1ggwK5UY3PagJrwZ0...
插入明文：'orange'，计算位置：13，密文：DHfCxAWG3TYfQMSGZQM4YHCjZ09vU/...

=====
客户端本地表 (local_table) 统计：
=====
'apple': 出现 7 次
'banana': 出现 4 次
'cherry': 出现 2 次
'orange': 出现 2 次
'pear': 出现 1 次

```

图 8: client.py 运行结果：混合插入与范围搜索

5.5 重复插入同一数值的测试

客户端增加了重复插入测试，连续 5 次插入 'apple'，再插入 3 次 'banana'：

```

1 def TestRepeatedInsert():
2     print("测试：连续插入相同数值 'apple' 5次")
3     for i in range(5):
4         print(f"--- 第 {i+1} 次插入 'apple' ---")
5         Insert('apple')
6
7     print("测试：连续插入相同数值 'banana' 3次")
8     for i in range(3):
9         print(f"--- 第 {i+1} 次插入 'banana' ---")
10        Insert('banana')

```

Listing 8: TestRepeatedInsert 重复插入测试

```
=====
客户端本地表 (local_table) 统计:
=====
'apple': 出现 7 次
'banana': 出现 4 次
'cherry': 出现 2 次
'orange': 出现 2 次
'pear': 出现 1 次

搜索范围: 'b' ~ 'p' (位置 7 ~ 15)
搜索结果:
密文: hfkdTJaC0c0XkLs2ZJDup8tY+pwZBN... -> 明文: banana
密文: toCHDiLxREBWJIZLunViiFaPDEMnz... -> 明文: banana
密文: D3HgEKRAuKwnbNUUXfkNDcPB6stL9u... -> 明文: banana
密文: cLTm8KwCWEETJgsemCtoB59IghlFuB... -> 明文: banana
密文: Mp/29QFkJY7q2i4PAsmofyrUKTTmYe... -> 明文: orange
密文: n5sFN1LzguBpox1xED9qPed0l8UrHu... -> 明文: cherry
密文: BA9I2NV/jAMcd1ggwk5UY3PagJrwZ0... -> 明文: cherry
密文: DHfCxAWG3TYfQMSGZQN4YHCjZ09vU/... -> 明文: orange
```

图 9: 重复插入同一明文，观察随机位置分布与编码变化

实验结果验证了以下几点：

- **频率隐藏：**同一个 'apple' 每次插入的位置 (pos) 不同，对应编码各异，密文完全不同，攻击者无法从密文分布推断 'apple' 出现了 5 次。
- **保序性保持：**所有 'apple' 的编码都小于所有 'banana' 的编码，大小顺序与明文字典序 (apple < banana) 一致。
- **编码均匀分布：**Encode 始终取区间中点，相同明文的多个编码在叶节点区间内相对均匀地散开。
- **节点分裂（数据量足够时）：**当叶节点密文数达到 128 时触发 rebalance()，后 64 条密文移入新节点，父节点 child_num 随之更新，数据库中受影响的编码由 FHUpdate 同步。

频率隐藏验证：插入 5 次 'apple' 和 2 次 'banana' 后，数据库中有 7 条密文，7 个 encoding 值全部不同。攻击者仅凭密文无法区分哪 5 条来自 'apple'，哪 2 条来自 'banana'，两者频率比 5:2 的信息被完全隐藏。

6 算法复杂度

操作	时间复杂度	空间复杂度	说明
Insert	$O(\log_M N)$	$O(1)$	$M=128$ 为节点容量, N 为总密文数
Search	$O(\log_M N + k)$	$O(1)$	k 为结果条数
Recode	$O(K)$	$O(K)$	K 为涉及节点的密文总数
Rebalance	$O(M)$	$O(M)$	仅分裂当前叶节点

客户端只维护一张明文计数表, 其大小等于明文域的基数 (不同明文的种数), 与数据库总数据量无关, 空间开销极小。

7 安全性分析

7.1 频率隐藏性

传统 OPE 因为确定性加密, 同一个词每次出来的密文都一样, 攻击者只需统计频率就能做猜测。FH-OPE 从两个层面打破这一点: 一是每次插入使用全新随机 IV, AES 加密后密文每次都不同; 二是同一明文在编码树里被随机安置在不同位置, 对应的编码各异。攻击者面对的是一堆看似均匀分布、互不相同的密文, 无法从频率分布得出任何有效信息。

7.2 保序性

叶节点之间按明文顺序链接, 同一叶节点内的编码严格递增, 相邻叶节点的编码区间 $[lower_i, upper_i)$ 互不重叠且按顺序排列。因此任意两条密文的编码大小关系, 完全对应其明文的字典序大小关系。

7.3 前向安全性

编码是动态变化的——插入新数据可能触发 **Recode**, 改变部分历史编码。即使攻击者获取了某个时间点的数据库快照, 也无法将快照中的历史编码与当前编码关联, 从而无法推断新插入数据与历史数据的关系。

8 遇到的问题和解决

问题一：动态链接库编译失败

在 WSL 中编译时，需要安装 `libmysqlclient-dev` 才有 MySQL 的头文件，编译命令还需要加 `-shared -fPIC` 选项，编译产物必须手动拷贝到 `/usr/lib/mysql/plugin/` 目录才能被 MySQL 加载。

问题二：Recode 的触发和数据库同步

调试时发现 `FHInsert` 有时返回 0，这是 `Encode` 触发了 `Recode` 后的正常返回值，告知存储过程需要执行 `UPDATE` 来同步受影响的编码。存储过程通过判断返回值是否为 0 来决定是否执行批量更新。

问题三：pymysql 连接方式

WSL 中 MySQL 默认通过 Unix socket 监听，而非 TCP 端口。使用 `host='localhost'` 连接会被拒绝。解决方法是改用 Unix socket 参数：

```
1 conn = pymysql.connect(  
2     unix_socket='/var/run/mysqld/mysqld.sock',  
3     user='root', password='123456', database='test_db'  
4 )
```

9 总结

本实验按照教材 7.3.3 节的描述，完整复现了 FH-OPE 频率隐藏保序加密方案。从编码树的数据结构设计到 MySQL UDF 接口，从客户端随机加密到重复值插入测试，整个链路均已打通并验证。

核心收获：

1. 编码树以 `child_num` 数组代替传统 B+ 树的关键词索引，实现了基于“计数”的位置定位，是整个方案的骨架。
2. 区域编码（Recode）只影响局部节点，使树的平衡调整代价可控，不会因一次插入引发全局更新。
3. CalPos 的随机性是频率隐藏的根源：同一明文在编码树中的插入位置是随机的，对应编码各不相同，攻击者无法通过统计密文频率猜测明文频率。
4. MySQL UDF + 存储过程的设计将编码逻辑完全封装在服务端，客户端只需维护轻量级的本地计数表，交互接口简洁高效。

附录：运行说明

环境要求

- Linux 环境（Ubuntu 22.04 或兼容发行版）

- MySQL 8.0+, 并已安装 libmysqlclient-dev
- Python 3.8+, 已安装 pycryptodome、pymysql
- OpenSSL (编译 -lcrypto 依赖)

完整部署步骤

1. 安装依赖: `sudo apt install mysql-server libmysqlclient-dev python3-pycryptodome python3-pymysql`
2. 启动 MySQL: `sudo service mysql start`
3. 编译: `g++ -shared -fPIC UDF.cpp Node.cpp -lcrypto -o libfhope.so`
4. 部署: `sudo cp libfhope.so /usr/lib/mysql/plugin/`
5. 初始化数据库: 进入 MySQL 后执行 `CREATE DATABASE test_db; USE test_db; SOURCE example.sql;`
6. 运行客户端: `python3 client.py`

文件清单

- Node.h —节点类声明及常量定义
- Node.cpp —编码树核心实现
- UDF.cpp —MySQL UDF 接口
- client.py —客户端程序 (含重复值测试)
- example.sql —数据库初始化脚本
- client_demo.py —不依赖 MySQL 的独立演示程序